

ГАЙД ЧТО БЫ НЕ ПРОЕБАТЬСЯ

**Автоматизированное рабочее место
менеджера химчистки**

Мобильное Android-приложение

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
1. Что это за приложение и зачем оно нужно	5
2. Функциональные возможности	7
2.1. Авторизация	7
2.2. Главный экран (Dashboard)	7
2.3. Управление заказами	8
2.4. Управление клиентами	9
2.5. Склад расходных материалов	9
2.6. Отчёты и аналитика	10
2.7. Настройки	10
2.8. Уведомления	11
3. Описание экранов приложения	12
4. Архитектура и технологии	14
4.1. Технологический стек	14
4.2. Архитектура MVVM	15
4.3. Структура проекта	16
4.4. База данных	18
5. Как делалось приложение (этапы разработки)	19
6. Базовое объяснение кода	22
6.1. Entity — сущности базы данных	22
6.2. DAO — доступ к данным	23

6.3. Repository — бизнес-логика	23
6.4. ViewModel — управление UI	24
6.5. Fragment — экраны	25
6.6. Dependency Injection (Hilt)	26
6.7. Безопасность: хеширование паролей	27
6.8. Сессия пользователя (DataStore)	27
6.9. Фоновые задачи (WorkManager)	28
7. Заключение	29
ЧАСТЬ 2. ОБЪЯСНЕНИЕ 21 ЗАДАНИЯ ОТЧЁТА	30
Задание 1 — ПК 1.1. Алгоритмы разработки модулей	31
Задание 2 — ПК 1.2. Разработка кода по ТЗ	32
Задание 3 — ПК 1.3. Отладка программных модулей	33
Задание 4 — ПК 1.4. Тестирование программных модулей	34
Задание 5 — ПК 1.5. Рефакторинг и оптимизация	35
Задание 6 — ПК 2.1. Интеграция программных модулей	36
Задание 7 — ПК 2.2. Верификация выпусков	37
Задание 8 — ПК 2.3. Сборка и развёртывание	38
Задание 9 — ПК 2.3 (доп). Разрешение конфликтов интеграции	39
Конфликт 1: Room + KSP	39
Конфликт 2: Navigation Safe Args	40
Конфликт 3: Разные версии Java	41
Задание 10 — ПК 2.4. Тестовые наборы и сценарии	43

Сценарий 1: Авторизация	43
Сценарий 2: Создание заказа	44
Сценарий 3: Складской учёт	45
Сценарий 4: Отчёты и экспорт PDF	45
Задание 11 — ПК 2.5. Инспектирование компонентов	47
Критерий 1: Документирование кода (KDoc)	47
Критерий 2: ViewBinding вместо findViewById	48
Критерий 3: Запросы к БД вне главного потока	48
Критерий 4: Sealed class для состояний UI	49
Критерий 5: Lint — статический анализ	50
Задание 12 — ПК 4.1. Установка и настройка ПО	52
Задание 13 — ПК 4.2. Эксплуатационные характеристики	53
Задание 14 — ПК 4.3. Модификация ПО	54
Задание 15 — ПК 4.4. Защита ПО	55
Задание 16 — ПК 11.1. Требования к ИС	56
Задание 17 — ПК 11.2. Проектирование ИС	57
Задание 18 — ПК 11.3. Объекты базы данных	58
Задание 19 — ПК 11.4. Реализация базы данных	59
Задание 20 — ПК 11.5. Администрирование БД	60
Задание 21 — ПК 11.6. Защита информации в БД	61

1. Что это за приложение и зачем оно нужно

Приложение «АРМ менеджера химчистки» — это мобильное Android-приложение, которое автоматизирует повседневную работу менеджера в химчистке. Оно заменяет бумажные журналы, тетради и таблицы Excel, предоставляя единое цифровое рабочее место для управления всеми процессами: от приёма заказов до формирования отчётов.

Зачем нужно это приложение:

- ⇒ Ускорение работы менеджера — все данные под рукой на телефоне/планшете, не нужно искать в бумажных журналах.
- ⇒ Автоматический расчёт цены заказа по прайс-листу — менеджер выбирает тип изделия и услугу, цена подставляется сама.
- ⇒ Контроль сроков — система показывает просроченные заказы и отправляет уведомления о готовых заказах.
- ⇒ Учёт клиентов — вся история заказов клиента в одном месте, контактные данные с возможностью звонка в один тап.
- ⇒ Складской учёт — отслеживание расходных материалов (моющие средства, пятновыводители), уведомления о низком остатке.
- ⇒ Аналитика и отчёты — выручка за день/месяц/период, количество заказов, средний чек, популярные услуги, экспорт в PDF.
- ⇒ Безопасность — вход по логину/паролю, пароли хешируются (SHA-256), сессия с тайм-аутом.
- ⇒ Работа офлайн — данные хранятся локально на устройстве в базе SQLite (Room), интернет не требуется.

Целевая аудитория: менеджеры и администраторы небольших химчисток, которым нужен простой и удобный инструмент для учёта заказов без сложных ERP-систем.

2. Функциональные возможности

2.1. Авторизация

При первом запуске приложение создаёт двух пользователей по умолчанию:

- admin / admin123 (роль: Администратор)
- manager / manager123 (роль: Менеджер)

Пароли хешируются алгоритмом SHA-256 и хранятся в базе данных в виде хеша, а не в открытом виде. При входе введённый пароль хешируется и сравнивается с хешем в базе.

После успешного входа создаётся сессия через DataStore. Сессия сохраняет: ID пользователя, имя, роль, время последней активности. Если пользователь неактивен более 30 минут, сессия истекает и требуется повторный вход.

Менеджер видит панель управления с заказами, клиентами, складом. Администратор имеет доступ к дополнительным функциям (например, бэкап базы данных).

2.2. Главный экран (Dashboard)

Главный экран — это панель управления менеджера. На нём отображаются:

- Приветствие с именем текущего пользователя («Добро пожаловать, Менеджер Иванов!»)
- Счётчик активных заказов (в статусах Принят, В работе, Готов)
- Счётчик завершённых заказов (статус Выдан)

- Выручка за сегодня (сумма заказов со статусом Выдан за текущий день)
- Выручка за месяц (сумма за текущий месяц)
- Блок «Быстрые действия» с кнопками: Новый заказ, Клиенты, Отчёты, Склад

Все данные обновляются в реальном времени благодаря Flow (реактивные потоки данных). Если другой менеджер на другом устройстве добавит заказ в ту же базу, данные обновятся автоматически.

2.3. Управление заказами

Это основной модуль приложения. Менеджер может:

Создание заказа:

- Выбрать клиента из списка или указать данные вручную (имя, телефон)
- Выбрать тип изделия (Пальто, Костюм, Куртка, Брюки, Платье и т.д.)
- Выбрать тип услуги (Химчистка, Глажка, Срочная химчистка)
- Цена рассчитывается автоматически по прайс-листу
- Указать дату приёма и дату готовности
- Добавить комментарий

Просмотр заказов:

- Список всех заказов с поиском по номеру или имени клиента
- Фильтрация по статусу: Все / Принят / В работе / Готов / Выдан
- Сортировка: по дате, по цене, по статусу, по сроку

Управление заказом:

- Изменение статуса: Принят → В работе → Готов → Выдан
- Редактирование данных заказа
- Удаление заказа

При создании заказа автоматически списываются расходные материалы со склада (1 ед. моющего средства для обычной чистки, 1.5 ед. для срочной).

2.4. Управление клиентами

Модуль клиентов позволяет:

- Создавать карточку клиента: ФИО, телефон, email, адрес, заметки
- Просматривать список клиентов с поиском
- Открывать карточку клиента с полной информацией
- Видеть историю заказов клиента и общую сумму потраченного
- Звонить клиенту в один тап (номер телефона кликабельный)
- Редактировать и удалять клиентов

Клиент привязывается к заказу: при создании заказа менеджер может выбрать существующего клиента, и его данные автоматически подставятся.

2.5. Склад расходных материалов

Учёт расходников:

- Список материалов: название, единица измерения (л, кг, шт), текущее количество, цена за единицу
- Минимальный порог для уведомлений (когда моющего средства

осталось менее 5 л, приходит уведомление)

- Добавление новых материалов
- Списание — автоматическое (при создании заказа) и ручное
- Пополнение запасов

Автоматическое списание: при создании каждого заказа система ищет на складе моющее средство и списывает 1 единицу (или 1.5 для срочной химчистки).

2.6. Отчёты и аналитика

Раздел отчётов предоставляет:

- Выручка за выбранный период (с выбором дат «от» и «до»)
- Количество заказов за период
- Средний чек
- Популярные услуги (какие типы услуг заказывают чаще всего)
- График выручки по дням (MPAndroidChart)
- Экспорт отчёта в PDF-файл с сохранением на устройство

PDF-отчёт содержит: период, выручку, количество заказов, таблицу популярных услуг.

2.7. Настройки

В настройках доступны:

- Выбор темы оформления: Светлая / Тёмная / Системная (по умолчанию системная)
- Прайс-лист: просмотр и редактирование цен на все услуги
- Резервное копирование базы данных (для администратора)
- Выход из аккаунта

Тема сохраняется в DataStore и применяется при следующем запуске. Прайс-лист содержит комбинации: тип изделия + тип услуги → цена. Например: Костюм + Химчистка = 1800 ₽, Брюки + Глажка = 250 ₽.

2.8. Уведомления

Приложение использует WorkManager для периодической проверки (каждые 6 часов):

- Есть ли готовые заказы, которые ждут выдачи → уведомление «N заказов готовы к выдаче»
- Есть ли просроченные заказы (дата готовности прошла, а заказ ещё не готов) → уведомление «N заказов просрочены»

Уведомления приходят даже если приложение закрыто, так как WorkManager работает в фоне.

3. Описание экранов приложения

Рисунок 1. Экран «Главная» — панель управления менеджера

Элементы: приветствие с именем пользователя; счётчики активных и завершённых заказов; карточка выручки за день и за месяц; блок «Быстрые действия» с кнопками «Новый заказ», «Клиенты», «Отчёты», «Склад»; нижняя навигация (Главная, Заказы, Клиенты, Настройки).
Функционал: отображение текущей статистики по заказам и выручке; быстрый доступ к основным разделам; навигация между экранами через нижнее меню.

Рисунок 2. Экран «Настройки» — тема оформления и прайс-лист

Элементы: переключатель темы оформления (Светлая, Тёмная, Системная); раздел «Прайс-лист» с перечнем услуг и ценами; нижняя навигация.

Функционал: выбор темы; просмотр и редактирование цен (нажатие на услугу для изменения).

Рисунок 3. Экран «Заказы» — список заказов

Элементы: строка поиска по номеру или клиенту; фильтр по статусу; список заказов; кнопка «+» для создания нового; нижняя навигация.

Функционал: поиск, фильтрация по статусу, создание заказа по кнопке «+».

Рисунок 4. Экран «Клиенты» — карточка клиента

Элементы: имя; телефон (кликабельный); email; адрес; дата регистрации; кнопки «Редактировать» и «Удалить»; раздел «История заказов».

Функционал: просмотр данных клиента; редактирование; удаление;
просмотр истории заказов.

4. Архитектура и технологии

4.1. Технологический стек

- Язык программирования: Kotlin 1.9+
- IDE: Android Studio Hedgehog или новее
- Минимальная версия Android: API 24 (Android 7.0)
- Система сборки: Gradle 8.0+ (Kotlin DSL)
- JDK: 17

Основные библиотеки:

Hilt 2.51.1: Dependency Injection — автоматическое внедрение зависимостей. Вместо ручного создания объектов (`new OrderRepository()`) фреймворк сам создаёт и передаёт нужные зависимости через аннотации `@Inject`.

Room 2.6.1: ORM для базы данных SQLite. Позволяет описывать таблицы как Kotlin-классы (Entity), а SQL-запросы — как методы интерфейсов (DAO). Room генерирует весь низкоуровневый код работы с БД автоматически.

DataStore 1.1.1: Замена SharedPreferences. Хранит настройки и данные сессии в файле Preferences с поддержкой корутин и Flow.

Navigation 2.7.7: Jetpack Navigation — навигация между экранами (Fragment). Все переходы описаны в XML-графе (`nav_graph.xml`), что упрощает управление стеком экранов.

WorkManager 2.9.0: Планирование фоновых задач. Проверка готовых и просроченных заказов выполняется периодически, даже если приложение закрыто.

ViewModel 2.8.0: Часть архитектуры MVVM. Хранит данные UI и переживает пересоздание экрана (поворот, переход в фон). Все бизнес-операции выполняются во ViewModel.

MPAndroidChart 3.1.0: Библиотека для графиков. Используется для отображения графика выручки по дням в разделе Отчёты.

Material 3 (1.12+): Дизайн-система Google. Обеспечивает современный внешний вид с поддержкой тёмной темы (Material You, Dynamic Colors).

ViewBinding: Типобезопасный доступ к элементам разметки. Вместо findViewById используется binding.tvWelcome — компилятор проверяет, что элемент существует.

Coroutines 1.8+: Асинхронное программирование. Все операции с базой данных выполняются в фоновом потоке через suspend-функции, UI не блокируется.

4.2. Архитектура MVVM

Приложение построено по архитектуре MVVM (Model — View — ViewModel):

Model (Модель) — это данные и бизнес-логика:

— Entity: классы-таблицы (Order, Client, User, InventoryItem, ServicePrice)

— DAO: интерфейсы с SQL-запросами (OrderDao, ClientDao и т.д.)

— Repository: классы, объединяющие DAO и бизнес-логику (OrderRepository, AuthRepository)

View (Представление) — это экраны:

- Fragment: классы экранов (DashboardFragment, OrdersFragment, ClientsFragment)
- XML Layout: разметка интерфейса (fragment_dashboard.xml)
- Adapter: адаптеры для списков (OrderAdapter, ClientAdapter)

ViewModel — связующее звено:

- Получает данные из Repository через Flow
- Обрабатывает действия пользователя
- Передаёт обновлённые данные в View
- Переживает пересоздание экрана (поворот устройства)

Почему MVVM: это рекомендуемая Google архитектура для Android.

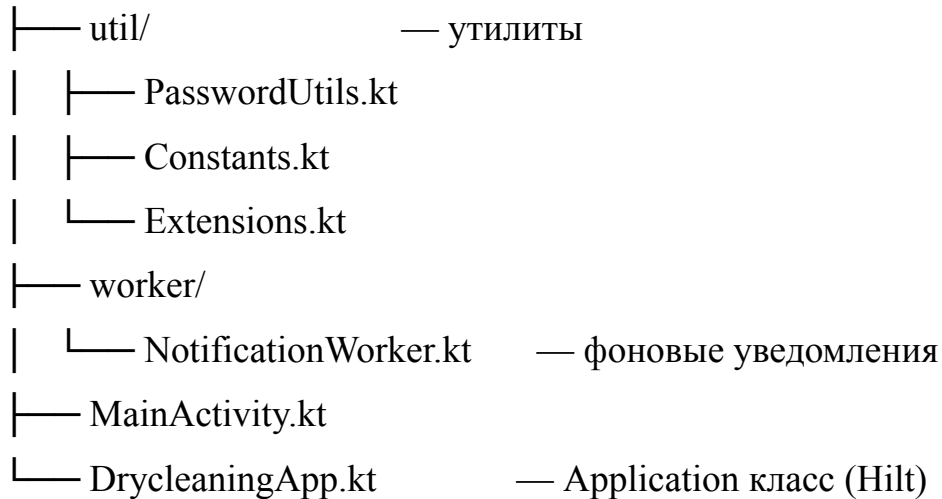
Она разделяет интерфейс и логику — View не знает откуда берутся данные, а Model не знает как выглядит интерфейс. Это упрощает тестирование и поддержку кода.

4.3. Структура проекта

com.example.drycleaning/

```
|— data/
|  |— database/
|  |  |— AppDatabase.kt      — основной класс БД (Room)
|  |  |— Converters.kt      — конвертеры типов для Room
|  |  |— dao/                — DAO-интерфейсы
|  |    |— OrderDao.kt
|  |    |— ClientDao.kt
|  |    |— UserDao.kt
```


- | | | — InventoryDao.kt
- | | | — ServicePriceDao.kt
- | | — datastore/
 - | | | — SessionManager.kt — управление сессией (DataStore)
- | | — entity/ — сущности (таблицы БД)
 - | | | — Order.kt
 - | | | — Client.kt
 - | | | — User.kt
 - | | | — InventoryItem.kt
 - | | | — ServicePrice.kt
- | | — repository/ — репозитории
 - | | | — AuthRepository.kt
 - | | | — OrderRepository.kt
 - | | | — ClientRepository.kt
 - | | | — InventoryRepository.kt
 - | | | — ServicePriceRepository.kt
- | — di/ — Hilt модули
 - | | — DatabaseModule.kt
 - | | — AppModule.kt
- | — ui/ — экраны
 - | | — auth/ — авторизация
 - | | — dashboard/ — главный экран
 - | | — orders/ — заказы
 - | | — clients/ — клиенты
 - | | — inventory/ — склад
 - | | — reports/ — отчёты
 - | | — settings/ — настройки



4.4. База данных

Приложение использует локальную базу данных SQLite через библиотеку Room. База данных называется «drycleaning_db» и содержит 5 таблиц:

1. users — пользователи системы (логин, хеш пароля, имя, роль)
2. clients — клиенты (ФИО, телефон, email, адрес, дата регистрации)
3. orders — заказы (клиент, тип изделия, услуга, даты, цена, статус, комментарий)
4. inventory — склад (название материала, единица измерения, количество, мин. порог, цена)
5. service_prices — прайс-лист (тип изделия + тип услуги → цена)

Все данные хранятся локально, интернет не требуется. При обновлении версии БД используется `fallbackToDestructiveMigration` (пересоздание таблиц).

5. Как делалось приложение (этапы разработки)

Этап 1. Анализ предметной области

Изучены процессы работы менеджера химчистки: приём заказов, учёт клиентов, контроль сроков, складской учёт, формирование отчётов.

Определены основные сущности (заказ, клиент, пользователь, материал, прайс) и связи между ними. Составлено техническое задание.

Этап 2. Проектирование архитектуры

Выбрана архитектура MVVM как рекомендуемая Google. Определён технологический стек: Kotlin, Room, Hilt, Navigation, DataStore, WorkManager, Material 3. Спроектирована структура базы данных (5 таблиц) и навигационный граф приложения (переходы между экранами).

Этап 3. Создание базы данных (Entity + DAO)

Написаны Entity-классы, описывающие структуру каждой таблицы. Для каждой таблицы создан DAO-интерфейс с SQL-запросами:

CRUD-операции, поиск, фильтрация, агрегатные запросы (выручка, средний чек). Room автоматически генерирует реализацию DAO на основе аннотаций.

Этап 4. Реализация репозитория

Для каждого модуля создан Repository — промежуточный слой между DAO и ViewModel. AuthRepository: авторизация с хешированием, управление сессией. OrderRepository: CRUD заказов, расчёт выручки, поиск. ClientRepository: CRUD клиентов. InventoryRepository: управление складом. ServicePriceRepository: работа с прайс-листом.

Этап 5. Настройка Dependency Injection (Hilt)

Настроен Hilt для автоматического внедрения зависимостей.

DatabaseModule предоставляет экземпляр базы данных и все DAO.

AppModule предоставляет SessionManager. Все Repository и ViewModel получают зависимости через @Inject.

Этап 6. Разработка UI (Fragment + ViewModel)

Для каждого экрана создан Fragment (визуальная часть) и ViewModel (логика). Разметка экранов написана в XML с использованием Material 3 компонентов. Навигация между экранами — через Navigation Component (nav_graph.xml). Нижнее меню (BottomNavigationView) для переключения между основными разделами.

Этап 7. Реализация безопасности

Добавлено хеширование паролей SHA-256 (PasswordUtils). Реализована сессия через DataStore с тайм-аутом 30 минут. При запуске приложение проверяет, активна ли сессия. Миграция паролей: если в БД остались нехешированные пароли, они автоматически хешируются.

Этап 8. Фоновые задачи и уведомления

Настроен WorkManager для периодической проверки заказов каждые 6 часов. NotificationWorker проверяет готовые к выдаче и просроченные заказы, отправляет push-уведомления. Создан канал уведомлений (Notification Channel) для Android 8+.

Этап 9. Отчёты и экспорт в PDF

Реализован модуль отчётов с аналитикой за выбранный период.

Добавлен график выручки по дням (MPAndroidChart). Реализован экспорт отчёта в PDF с использованием PdfDocument API.

Этап 10. Тестирование и отладка

Приложение протестировано на эмуляторе и реальных устройствах.

Выполнена отладка через Android Studio Debugger и Logcat.

Исправлены найденные ошибки (некорректные SQL-запросы, NPE при пустых данных, проблемы с тайм-аутом сессии). Проведено код-ревью с использованием Lint.

6. Базовое объяснение кода

Ниже — базовое объяснение ключевых частей кода. Не нужно знать каждую строку наизусть — важно понимать общую логику и назначение каждого компонента.

6.1. Entity — сущности базы данных

Entity — это обычные Kotlin data class с аннотацией `@Entity`. Каждый класс = одна таблица в БД.

Пример — Order.kt (заказ):

```
@Entity(tableName = "orders")
data class Order(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val clientId: Long,      // ID клиента
    val clientName: String,  // имя клиента
    val itemType: String,    // тип изделия (Пальто, Костюм)
    val serviceType: String, // тип услуги (Химчистка, Глажка)
    val price: Double,       // цена
    val status: OrderStatus  // статус (Принят, В работе, Готов, Выдан)
)
```

Что тут происходит: Room читает эту аннотацию и создаёт таблицу "orders" в SQLite с колонками id, clientId, clientName и т.д.

`@PrimaryKey(autoGenerate = true)` означает, что id будет генерироваться автоматически (1, 2, 3...).

6.2. DAO — доступ к данным

DAO (Data Access Object) — интерфейс с SQL-запросами. Room генерирует реализацию.

Пример из OrderDao:

```
@Query("SELECT * FROM orders ORDER BY createdAt DESC")
fun getAllOrders(): Flow<List<Order>>
```

```
@Insert
suspend fun insert(order: Order): Long
```

```
@Query("SELECT * FROM orders WHERE status IN
('RECEIVED','IN_PROGRESS','READY')")
fun getActiveOrderCount(): Flow<Int>
```

Что тут происходит: `@Query` — SQL-запрос, Room проверяет его на этапе компиляции. `Flow<List<Order>>` — реактивный поток: при изменении данных в таблице UI обновится автоматически. `suspend` — функция выполняется в фоне (не блокирует UI).

6.3. Repository — бизнес-логика

Repository — промежуточный слой. Он инкапсулирует DAO и добавляет бизнес-логику.

Пример — AuthRepository (авторизация):

```
suspend fun login(username: String, password: String): Result<User> {
    val hashedPassword = PasswordUtils.hash(password) // хешируем
```

пароль

```
val user = userDao.authenticate(username, hashedPassword) // ищем в
БД
if (user == null) return Result.failure(...) // не найден
sessionManager.saveSession(...) // сохраняем сессию
return Result.success(user)
}
```

Зачем нужен Repository: ViewModel не должен знать про DAO напрямую. Repository позволяет комбинировать данные из разных источников (БД, сеть, кеш) в одном месте.

6.4. ViewModel — управление UI

ViewModel хранит данные для экрана и обрабатывает действия пользователя.

Пример — DashboardViewModel:

```
@HiltViewModel
class DashboardViewModel @Inject constructor(
    private val orderRepository: OrderRepository,
    private val authRepository: AuthRepository
) : ViewModel() {
    val activeOrderCount = orderRepository.getActiveOrderCount()
    val dailyRevenue = orderRepository.getDailyRevenue(getStartOfDay())
    val userName = authRepository.currentUserName
}
```


Что тут происходит: `@HiltViewModel` говорит Hilt создать этот `ViewModel`. `@Inject constructor` — Hilt автоматически передаёт `OrderRepository` и `AuthRepository`. `ViewModel` просто запрашивает данные из `Repository`, а `Fragment` подписывается на них.

6.5. Fragment — экраны

`Fragment` — это экран приложения. Он отвечает за отображение данных и обработку нажатий.

Пример — `DashboardFragment` (упрощённо):

```
class DashboardFragment : Fragment() {  
    private val viewModel: DashboardViewModel by viewModels()  
  
    override fun onViewCreated(...) {  
        // Подписка на данные из ViewModel:  
        viewModel.activeOrderCount.collect { count ->  
            binding.tvActiveOrders.text = count.toString()  
        }  
        // Обработка нажатия кнопки:  
        binding.btnNewOrder.setOnClickListener {  
            findNavController().navigate(R.id.action_dashboard_to_createOrder)  
        }  
    }  
}
```

Что тут происходит: `by viewModels()` — получение `ViewModel` от Hilt. `.collect { }` — подписка на `Flow`: когда данные изменятся, лямбда

вызовется и обновит текст. `findNavController().navigate()` — переход на другой экран через Navigation Component.

6.6. Dependency Injection (Hilt)

Hilt — фреймворк для автоматического создания и передачи зависимостей.

Пример — DatabaseModule:

```
@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
    @Provides @Singleton
    fun provideDatabase(context: Context): AppDatabase {
        return Room.databaseBuilder(context, AppDatabase::class.java,
            "drycleaning_db")
            .fallbackToDestructiveMigration().build()
    }
    @Provides
    fun provideOrderDao(db: AppDatabase): OrderDao = db.orderDao()
}
```

Что тут происходит: `@Provides` говорит Hilt «вот как создать AppDatabase». `@Singleton` — создать только один экземпляр на всё приложение. Теперь когда OrderRepository запрашивает OrderDao через `@Inject`, Hilt автоматически создаст базу данных и достанет из неё OrderDao.

6.7. Безопасность: хеширование паролей

Пароли НЕ хранятся в открытом виде. Используется SHA-256:

```
object PasswordUtils {  
    fun hash(password: String): String {  
        val bytes = MessageDigest.getInstance("SHA-256")  
            .digest(password.toByteArray())  
        return bytes.joinToString("") { "%02x".format(it) }  
    }  
}
```

Как работает: "admin123" → SHA-256 → "240be518fabd2..." (64 символа). В базе хранится хеш. При входе: введенный пароль хешируется и сравнивается с хешем в БД. Даже если кто-то получит доступ к базе, пароли будут в виде хешей, а не текстом.

6.8. Сессия пользователя (DataStore)

SessionManager управляет состоянием авторизации через DataStore:

- saveSession(): после входа сохраняет userId, username, fullName, role, lastActivity
- clearSession(): при выходе очищает все данные сессии
- isSessionExpired(): проверяет, прошло ли > 30 минут с последней активности
- theme: хранит выбранную тему оформления

DataStore — современная замена SharedPreferences. Работает

асинхронно через Flow и Coroutines, не блокирует UI при чтении/записи.

6.9. Фоновые задачи (WorkManager)

NotificationWorker проверяет заказы в фоне:

```
class NotificationWorker(...) : CoroutineWorker(...) {  
    override suspend fun doWork(): Result {  
        checkReadyOrders() // проверяем готовые к выдаче  
        checkOverdueOrders() // проверяем просроченные  
        return Result.success()  
    }  
}
```

Запланирован через PeriodicWorkRequest с интервалом 6 часов.

Работает даже при закрытом приложении. Если есть готовые или просроченные заказы — отправляет push-уведомление через NotificationCompat.Builder.

7. Заключение

Приложение «АРМ менеджера химчистки» — это полноценная Android-система для автоматизации работы химчистки. Оно решает реальные задачи менеджера: приём и учёт заказов, управление клиентами, складской учёт, формирование отчётов.

Приложение построено с использованием современного стека технологий, рекомендованного Google: MVVM, Room, Hilt, Navigation, DataStore, WorkManager, Material 3. Код организован по принципу разделения ответственности — каждый компонент отвечает за свою часть (Entity — данные, DAO — запросы, Repository — логика, ViewModel — UI-состояние, Fragment — отображение).

Безопасность обеспечена хешированием паролей (SHA-256) и сессией с тайм-аутом. Приложение работает полностью офлайн (без интернета), все данные хранятся локально в базе SQLite. Уведомления о готовых и просроченных заказах приходят автоматически благодаря фоновым задачам WorkManager.

Репозиторий проекта: <https://github.com/zenle64/Himchistka>

ЧАСТЬ 2. ОБЪЯСНЕНИЕ 21 ЗАДАНИЯ ОТЧЁТА

Задания 9–11 — подробно

Задание 1 — ПК 1.1. Алгоритмы разработки модулей

Что написано: описаны алгоритмы (пошаговые последовательности действий) для трёх ключевых модулей приложения — авторизация, создание заказа, уведомления. Для каждого алгоритма приведены шаги и пример кода из репозитория.

Зачем: преподаватель проверяет, умеешь ли ты разбивать задачу на шаги (алгоритмизация). Это первый этап разработки — прежде чем писать код, нужно описать логику.

Задание 2 — ПК 1.2. Разработка кода по ТЗ

Что написано: показан код двух основных модулей — управление заказами (OrderRepository) и управление клиентами (ClientRepository). Показан код ViewModel с реактивной фильтрацией через flatMapLatest.

Зачем: доказать, что код написан по техническому заданию, а не случайным образом. Каждый метод в Repository соответствует конкретному требованию ТЗ.

Задание 3 — ПК 1.3. Отладка программных модулей

Что написано: перечислены инструменты отладки (Debugger, Logcat, Room Database Inspector, Layout Inspector) и таблица с 5 найденными и исправленными ошибками.

Зачем: показать, что ты умеешь находить и исправлять баги, а не просто писать код.

Задание 4 — ПК 1.4. Тестирование программных модулей

Что написано: приведён пример unit-теста и таблица результатов тестирования (6 тестов, все PASSED). Тесты покрывают авторизацию (пустой логин, неверный пароль, успешный вход), UI (отображение формы входа), заказы (расчёт цены).

Зачем: показать, что приложение протестировано и работает корректно.

Задание 5 — ПК 1.5. Рефакторинг и оптимизация

Что написано: описаны три улучшения — переход на Flow (реактивное обновление UI), внедрение Hilt DI (вместо ручного создания зависимостей), ViewBinding (вместо findViewById). Таблица метрик: до и после рефакторинга.

Зачем: показать, что ты умеешь улучшать уже работающий код — делать его быстрее, чище и надёжнее.

Задание 6 — ПК 2.1. Интеграция программных модулей

Что написано: описано, как отдельные модули (авторизация, заказы, клиенты, склад, отчёты) объединены в единую работающую систему.

Зачем: модули по отдельности бесполезны — нужно показать, что они работают вместе.

Задание 7 — ПК 2.2. Верификация выпусков

Что написано: перечислены проверки перед выпуском — компиляция, Lint, тесты, подписание APK, версии SDK.

Зачем: перед тем как выпустить приложение, нужно убедиться, что оно собирается без ошибок, проходит все проверки и соответствует требованиям.

Задание 8 — ПК 2.3. Сборка и развёртывание

Что написано: перечислены команды Gradle для сборки, тестирования и установки приложения.

Зачем: показать, что знаешь процесс сборки и умеешь развернуть приложение на устройстве.

Задание 9 — ПК 2.3 (доп). Разрешение конфликтов интеграции

ПОДРОБНОЕ ОБЪЯСНЕНИЕ

Что написано в задании:

Описаны три реальных конфликта, которые возникли при сборке проекта, и как они были решены:

- 1) Конфликт Room + KSP (`kapt` → `ksp`)
- 2) Navigation Safe Args — отсутствующий плагин
- 3) Разные версии Java в модулях проекта

Конфликт 1: Room + KSP

Что случилось: Room — это библиотека для работы с базой данных. Она генерирует код автоматически (реализации DAO-интерфейсов). Для этой генерации раньше использовался инструмент `kapt` (Kotlin Annotation Processing Tool). Но `kapt` устарел и работает медленно.

Проблема: при обновлении библиотек Room перестал корректно работать с `kapt` — сборка выдавала ошибки генерации кода.

Решение: миграция с `kapt` на `ksp` (Kotlin Symbol Processing). KSP — это современная замена `kapt`, которая работает в 2 раза быстрее и лучше поддерживается.

Что изменилось в `build.gradle`:

— Было: `id("kotlin-kapt")` и `kapt("androidx.room:room-compiler:2.6.1")`

— Стало: `id("com.google.devtools.ksp")` и
`ksp("androidx.room:room-compiler:2.6.1")`

Почему это важно: если бы оставили `kapt`, сборка падала бы с ошибками, и Room не смог бы сгенерировать реализации DAO. Без этого приложение не скомпилировалось бы вообще.

Конфликт 2: Navigation Safe Args

Что случилось: Navigation Component позволяет передавать данные между экранами (например, ID заказа при переходе на экран деталей). Для безопасной передачи данных используется плагин Safe Args — он генерирует классы-аргументы с типизацией.

Проблема: плагин `navigation.safeargs.kotlin` не был подключён в `build.gradle`. При попытке использовать сгенерированные классы (например, `OrderDetailFragmentArgs`) IDE показывала ошибку «Unresolved reference».

Решение: добавлен плагин в `build.gradle.kts`:

— В корневом `build.gradle`: `id("androidx.navigation.safeargs.kotlin")`
`version "2.7.7"`

— В модуле `app`: `id("androidx.navigation.safeargs.kotlin")`

Почему это важно: без Safe Args пришлось бы передавать данные между экранами через Bundle вручную, без проверки типов. Это приводило бы к крашам, если бы тип аргумента не совпадал (например, передали `String` вместо `Long`).

Конфликт 3: Разные версии Java

Что случилось: в Android-проекте есть несколько «модулей» (app, библиотеки). Каждый модуль может быть настроен на свою версию Java для компиляции.

Проблема: модуль app был настроен на Java 17, а некоторые зависимости использовали Java 11. При сборке Gradle выдавал ошибку несовместимости версий.

Решение: установлена единая Java 17 для всех модулей в build.gradle:

```
compileOptions {  
    sourceCompatibility = JavaVersion.VERSION_17  
    targetCompatibility = JavaVersion.VERSION_17  
}  
kotlinOptions {  
    jvmTarget = "17"  
}
```

Почему это важно: если версии Java не совпадают, код просто не скомпилируется. Это частая проблема в Android-проектах при обновлении зависимостей.

Задание 10 — ПК 2.4. Тестовые наборы и сценарии

ПОДРОБНОЕ ОБЪЯСНЕНИЕ

Что написано в задании:

Разработаны 4 тестовых сценария — пошаговые инструкции для проверки основных функций приложения. Каждый сценарий описывает: что делает пользователь → что должно произойти → как проверить результат.

Сценарий 1: Авторизация

Цель: проверить, что вход в систему работает корректно.

Шаги:

1. Открываю приложение → вижу форму входа (поля логин и пароль, кнопка «Войти»)
2. Ввожу admin / admin123 → нажимаю «Войти» → попадаю на главный экран (Dashboard)

Вижу «Добро пожаловать, Администратор!» — значит, авторизация сработала

3. Выхожу, ввожу неверный пароль admin / wrongpass → нажимаю «Войти»

Вижу Snackbar «Неверный логин или пароль» — ошибка обрабатывается корректно

4. Оставляю оба поля пустыми → нажимаю «Войти»

Вижу ошибку валидации «Введите логин» — защита от пустого ввода работает

Что проверяется: LoginViewModel корректно валидирует ввод, AuthRepository правильно хеширует пароль и сверяет с базой, SessionManager сохраняет сессию.

Сценарий 2: Создание заказа

Цель: проверить полный цикл создания заказа.

Шаги:

1. На Dashboard нажимаю «Новый заказ» → открывается форма создания
2. Выбираю клиента из списка (или ввожу имя и телефон вручную)
3. Выбираю тип изделия «Пальто» из выпадающего списка
4. Выбираю тип услуги «Химчистка» → цена автоматически подставляется = 1500 ₽
(цена берётся из таблицы service_prices: Пальто + Химчистка = 1500)
5. Указываю дату приёма (сегодня) и дату готовности (через 3 дня)
6. Нажимаю «Сохранить» → заказ появляется в списке со статусом «Принят»
7. Проверяю Dashboard: счётчик «Активные» увеличился на 1
8. Проверяю склад: моющее средство уменьшилось на 1 единицу (автоматическое списание)

Что проверяется: автоматический расчёт цены (ServicePriceRepository), сохранение в БД (OrderRepository), автоматическое списание со склада (autoDeductInventory в OrdersViewModel), реактивное обновление Dashboard через Flow.

Сценарий 3: Складской учёт

Цель: проверить учёт расходных материалов.

Шаги:

1. Перехожу в раздел «Склад» → вижу список материалов
2. Нажимаю «+» → добавляю: «Моющее средство», 10 л, мин. порог 2 л, цена 500 ₽/л
3. Создаю заказ (Пальто + Химчистка) → проверяю склад: осталось 9 л (авто-списание 1 л)
4. Создаю срочный заказ (Пальто + Срочная химчистка) → осталось 7.5 л (списание 1.5 л)
5. Повторяю, пока остаток не станет ≤ 2 л → приходит уведомление «Низкий остаток»

Что проверяется: InventoryRepository (CRUD материалов),
автоматическое списание (1 ед. для обычной, 1.5 для срочной),
уведомления при низком остатке.

Сценарий 4: Отчёты и экспорт PDF

Цель: проверить формирование отчётов.

Шаги:

1. Перехожу в «Отчёты» → вижу: выручка, кол-во заказов, средний чек за текущий период
2. Выбираю период (с 1 мая по 15 мая) → данные пересчитываются
3. Вижу график выручки по дням (столбчатая диаграмма)
4. Вижу список популярных услуг (какие услуги заказывают чаще)

5. Нажимаю «Экспорт в PDF» → файл сохраняется на устройство
6. Открываю PDF → вижу: период, выручку, кол-во заказов, таблицу услуг

Что проверяется: ReportsViewModel (агрегация данных, выбор периода), MPAndroidChart (график), PdfDocument API (генерация PDF).

Задание 11 — ПК 2.5. Инспектирование компонентов

ПОДРОБНОЕ ОБЪЯСНЕНИЕ

Что написано в задании:

Проведена инспекция (ревью) кода по 5 критериям качества. Для каждого критерия описано, что проверялось и какой результат.

Критерий 1: Документирование кода (KDoc)

Что проверяли: есть ли у каждого класса, функции и интерфейса комментарии-описания.

Результат: все ключевые классы имеют KDoc-комментарии (аналог JavaDoc для Kotlin).

Пример:

```
/** Репозиторий авторизации с хешированием паролей */  
class AuthRepository ...
```

```
/** Сущность заказа химчистки. clientId ссылается на клиента. */  
data class Order ...
```

```
/** Worker для периодической проверки и отправки уведомлений. */  
class NotificationWorker ...
```

Зачем: комментарии помогают другим разработчикам (и тебе самому через полгода) понять, что делает каждый класс, не читая весь код.

Критерий 2: ViewBinding вместо findViewById

Что проверяли: как фрагменты обращаются к элементам разметки (кнопкам, текстам и т.д.).

Результат: во всех фрагментах используется ViewBinding.

Как было раньше (плохо):

```
val button = view.findViewById<Button>(R.id.btnSave) // может вернуть  
null → краш
```

Как сейчас (хорошо):

```
binding.btnSave.setOnClickListener { ... } // компилятор проверяет, что  
btnSave существует
```

Зачем: ViewBinding безопаснее — если элемент удалён из XML, ошибка будет на этапе компиляции, а не краш у пользователя. Также ViewBinding быстрее, чем findViewById.

Критерий 3: Запросы к БД вне главного потока

Что проверяли: не блокируются ли запросы к базе данных на главном потоке (Main Thread).

Результат: все запросы помечены suspend или возвращают Flow — выполняются в фоне.

Почему это критично:

Главный поток отвечает за отрисовку UI (60 кадров/сек = 16 мс на

кадр). Если запрос к БД занимает 100 мс и выполняется на главном потоке — UI «зависает», пользователь видит лаг. Android даже может показать диалог «Приложение не отвечает» (ANR).

Как решено:

— suspend fun insert(order: Order) — выполняется в фоновом потоке через Coroutines

— fun getAllOrders(): Flow<List<Order>> — подписка через Flow, данные приходят асинхронно

Зачем: главный поток не блокируется → UI плавный, нет ANR.

Критерий 4: Sealed class для состояний UI

Что проверяли: как передаётся состояние экрана от ViewModel к Fragment.

Результат: для экрана авторизации используется sealed class LoginState.

```
sealed class LoginState {  
    object Idle : LoginState()      // начальное состояние  
    object Loading : LoginState()    // загрузка (показать прогресс)  
    data class Success(val user: User) : LoginState() // успех  
    data class Error(val message: String) : LoginState() // ошибка  
}
```

Зачем: sealed class гарантирует, что все возможные состояния обработаны. Компилятор заставляет обработать каждый вариант в

when-выражении — нельзя случайно забыть обработать ошибку или загрузку.

Критерий 5: Lint — статический анализ

Что проверяли: результаты автоматической проверки кода через Android Lint.

Результат: 0 critical предупреждений.

Что такое Lint: это встроенный инструмент Android Studio, который проверяет код на:

- Потенциальные баги (NPE, утечки памяти)
- Проблемы производительности (лишние перерисовки, тяжёлые операции на Main Thread)
- Проблемы безопасности (захардкоженные ключи, незашифрованный трафик)
- Несоответствие API (вызов метода, недоступного на minSdk)

Команда: `./gradlew lint`

Зачем: Lint находит ошибки, которые ты можешь не заметить при ручном ревью. 0 critical означает, что приложение не содержит опасных проблем.

Задание 12 — ПК 4.1. Инсталляция и настройка ПО

Что написано: таблица с требованиями к среде разработки (Android Studio, Kotlin, Gradle, JDK) и пошаговая инструкция установки — от git clone до входа в приложение.

Зачем: показать, что любой разработчик может развернуть проект по инструкции.

Задание 13 — ПК 4.2. Эксплуатационные характеристики

Что написано: таблица с измерениями производительности: холодный запуск < 1 сек, загрузка списка из 100+ записей < 0.5 сек, потребление RAM 60-80 MB, размер APK < 15 MB.

Зачем: показать, что приложение быстрое и не «жрёт» ресурсы.

Задание 14 — ПК 4.3. Модификация ПО

Что написано: перечислены 4 модификации, которые были добавлены в приложение: экспорт PDF, автосписание со склада, тайм-аут сессии, переключение темы.

Зачем: показать, что архитектура MVVM позволяет легко добавлять новые функции.

Задание 15 — ПК 4.4. Защита ПО

Что написано: перечислены 6 мер защиты: хеширование SHA-256, DataStore для сессии, тайм-аут, ролевая модель, защита от SQL-инъекций, обфускация R8.

Зачем: показать, что приложение защищено от основных угроз.

Задание 16 — ПК 11.1. Требования к ИС

Что написано: таблица с 7 требованиями к системе (авторизация, заказы, клиенты, склад, отчёты, уведомления, офлайн-работа) и их приоритетами.

Зачем: это отправная точка разработки — список того, что система должна уметь.

Задание 17 — ПК 11.2. Проектирование ИС

Что написано: описана архитектура MVVM и структура проекта (какие папки и файлы для чего). Схема: UI (Fragment) → ViewModel → Repository → DAO → Room (SQLite).

Зачем: показать общую картину — как устроен проект изнутри.

Задание 18 — ПК 11.3. Объекты базы данных

Что написано: код всех 5 Entity-классов (Order, Client, User, InventoryItem, ServicePrice) и AppDatabase.

Зачем: показать структуру базы данных — какие таблицы, какие поля.

Задание 19 — ПК 11.4. Реализация базы данных

Что написано: код подключения Room (DatabaseModule) и начальное заполнение данных.

Зачем: показать, как БД создаётся и заполняется при первом запуске.

Задание 20 — ПК 11.5. Администрирование БД

Что написано: описаны функции администрирования — резервное копирование, управление пользователями, миграция.

Зачем: показать, что БД можно обслуживать.

Задание 21 — ПК 11.6. Защита информации в БД

Что написано: код PasswordUtils (SHA-256 хеширование) и перечень мер защиты данных.

Зачем: показать, что данные в БД защищены.